# DELEQATLAR

Deleqatların əsas məqsədi **metodları obyekt kimi istifadə etmək** və onları parametr kimi ötürməkdir. Bu, C#-da daha elastik və modullu kod yazmağa imkan verir.

# Niyə İstifadə Edirik?

```
CalcDelegate del = Sum;
del += Substract;
del += Multiply;
del += Divide;
del(1,1);
```

Bir deleqat bir və ya bir neçə metodun “ünvanını” saxlayır və lazım olduqda onları çağırır.

```
public delegate void CalcDelegate(int a,int b);

0 references
static void Sum(int a, int b)
{
    Console.WriteLine(a+b);
}

0 references
static void Substract(int a, int b)
{
    Console.WriteLine(a - b);
}

0 references
static void Multiply(int a, int b)
{
    Console.WriteLine(a * b);
}

0 references
static void Divide(int a, int b)
{
    Console.WriteLine(a / b);
}
```

| İstifadə səbəbi | İzah |
| --- | --- |
| **Callback** | Bir metoda digər metodu parametr kimi ötürmək (iş bitəndə geri çağırmaq). |
| **Event sistemi** | event açar sözü deleqatın üzərində qurulub, bir çox abonentə xəbər vermək mümkündür. |
| **Dinamik metod çağırışı** | Hansı metodun işləyəcəyini runtime-da müəyyən etmək. |
| **Kodu ayırmaq və çeviklik** | Metod konkret əməliyyatı bilməsin, yalnız deleqat qəbul etsin. |

Bir metodun içinə digər metodun “ünvanını” ötürürsən. O metod öz işini görür, sonra sənin ötürdüyün metodu çağırır.
Bu, “işim bitəndə bu funksiyanı çağır” deməkdir.

```
void ProcessData(int x,Action<int> callback)
{
    int result = x * 2;
    callback(result);
}

ProcessData(5, result => Console.WriteLine($"Nəticə: {result}"));
```

# Deleqat Tipləri

- **Action** - geriyə dəyər qaytarmayan metodlar üçün:

```
Action<string> actionDelegate = Console.WriteLine;
actionDelegate("Salam!");

Action<string> action = message => Console.WriteLine(message);
action("Hello, World!");

Action<int, string, bool> complexAction = (id, name, isActive) =>
{
    Console.WriteLine($"ID: {id}, Ad: {name}, Aktiv: {isActive}");
};
```

# Func deleqatı-**Dəyər qaytaran metodlar üçün**

```
Func<int> Random = () => new Random().Next(1, 100);
Func<int, int, int> Add = (x, y) => x + y;
Add(5, 10);
```

**Func delegate** -return type olur və sonda 1 dəyər qaytarır. Nəticə qaytaran metodlar üçün. Son tip həmişə dönüş tipidir (Func<T1,T2,TResult>).Burda 2 int tipində dəyər qəbul edir və int tipinda nəticə qaytarır.

# Predicate<T>

- Bu, əslində **Func<T,bool>** kimi işləyir.
- Yəni bir T tipli parametr alır,bool qaytarır.
- Siyahı filterləməkdə, şərt yoxlamaqda çox işlənir.

```
Predicate<int> isEven = number => number % 2 == 0;

Console.WriteLine(isEven(4)); // True

Predicate<int> checkPositive = x => x > 0;
Console.WriteLine(checkPositive(-5)); // False
```

| Tip | Parametrlər | Dönüş tipi |
| --- | --- | --- |
| **Action** | 0–16 parametr | void |
| **Func** | 0–16 parametr | Son tip = dönüş tipi |
| **Predicate** | 1 parametr (T) | bool |